

System wspomagania decyzji w wycenie diamentów

Część II: Algorytmy uczenia maszynowego i ich interpretacja

Maryia Audzei, IST, sem 4

MSiD Lab 17:05 TN (grupa 6)

1. Wstęp

Celem drugiej części projektu jest przejście od ręcznie definiowanych reguł decyzyjnych (tworzonych w pierwszej części) do zastosowania klasycznych algorytmów uczenia maszynowego. O ile na poprzedniej liście przekonałam się, jak czasochłonne jest manualne tworzenie systemów „if/else” oraz szacowanie progów dla dużych zbiorów danych, o tyle w niniejszym raporcie skupiłam się na automatyzacji tego procesu.

Głównym zadaniem badawczym jest nie tylko wykorzystanie gotowych modeli, ale przede wszystkim zrozumienie ich podstaw matematycznych. Wymagało to zaimplementowania kluczowych algorytmów, takich jak Regresja Liniowa (metodą analityczną oraz spadku gradientu) i funkcji Zysku Informacyjnego od zera, a następnie zestawienia ich wyników z profesjonalną biblioteką scikit-learn. Dodatkowym, niezwykle istotnym aspektem jest ocena interpretowalności modeli, analiza wpływu skalowania danych na wagi modelu oraz badanie zjawisk takich jak kompromis obciążenie-wariancja (*Bias-Variance Tradeoff*) czy błąd ekstrapolacji w obliczu anomalii.

2. Zbiór danych i Preprocessing

Analiza kontynuowana jest na zbiorze **diamonds**, który był podstawą badań na Liście 1. Zbiór ten zawiera dane o ponad 50 tysiącach diamentów, w tym ich cechy numeryczne (masa, wymiary) oraz kategoryjne (jakość szlif, kolor, przejrzystość).

Ze względu na specyfikę algorytmów uczenia maszynowego, które wymagają danych wyłącznie w postaci liczbowej, zastosowano proces preprocessingu. Cechy kategoryjne przekształcono za pomocą metody kodowania *One-Hot Encoding* (funkcja `pd.get_dummies`), co pozwoliło na utworzenie zmiennych binarnych. Następnie zbiór danych podzielono na część treningową (80%) oraz testową (20%), gwarantując brak

wycieku informacji podczas ewaluacji algorytmów. Do zadanego problemu klasyfikacji przyjęto próg ceny > 5000 USD, określając w ten sposób klasę diamentów „drogich”.

3. Klasyfikacja za pomocą Drzew Decyzyjnych

W pierwszym etapie wykorzystano algorytm DecisionTreeClassifier z biblioteki scikit-learn do klasyfikacji danych. W celu uniknięcia przeuczenia oraz umożliwienia późniejszej analizy, ograniczono maksymalną głębokość drzewa do 5 poziomów ($\text{max_depth}=5$).

Metryki na zbiorze testowym:

```
Drzewo decyzyjne - Klasyfikacja
Accuracy na zbiorze testowym: 0.9565
F1-Score na zbiorze testowym: 0.9219
```

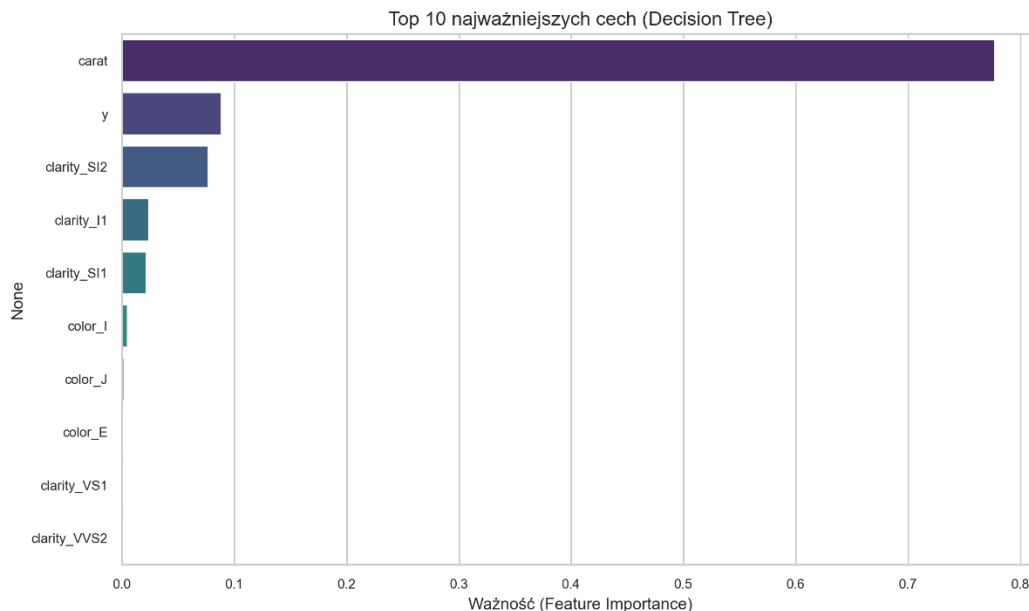
Wysoka wartość F1-Score na poziomie 92% jednoznacznie udowadnia, że model doskonale radzi sobie z klasyfikacją na niezbalansowanym zbiorze. W porównaniu z moimi manualnymi regułami z Listy 1, drzewo podejmuje bardziej stabilne decyzje, nie faworyzując klasy dominującej.

3.1. Interpretacja reguł i Zysk Informacyjny

Kluczowym elementem była analiza sposobu "myślenia" drzewa decyzyjnego. Z wygenerowanej struktury warunkowej jasno wynika, że głównym (pierwszym) węzłem podziału jest masa diamentu: `--- carat <= 1.00`. Następnie algorytm dzieli dane według wymiarów fizycznych (np. $y \leq 6.15$) oraz jakości (clarity).

```
>>> Interpretacja drzewa i ważność cech
Fragment struktury reguł drzewa:
|--- carat <= 1.00
| |--- y <= 6.15
| | |--- y <= 6.09
| | | |--- truncated branch of depth 3
| | |--- y > 6.09
| | | |--- truncated branch of depth 3
| |--- y > 6.15
| | |--- clarity_VS1 <= 0.50
| | | |--- truncated branch of depth 3
| | |--- clarity_VS1 > 0.50
| | | |--- truncated branch of depth 3
|--- carat > 1.00
| |--- y <= 6.76
| | |--- clarity_SI2 <= 0.50
| | | |--- truncated branch of depth 3
| | |--- clarity_SI2 > 0.50
| | | |--- truncated branch of depth 3
| |--- y > 6.76
| | |--- y <= 6.96
| | | |--- truncated branch of depth 3
| | |--- y > 6.96
| | | |--- truncated branch of depth 3
```

Analiza atrybutu `feature_importances_` wykazała, że masa diamentu (`carat`) odpowiada za aż 77,72% całej decyzyjności modelu. Pokrywa się to dokładnie z moją intuicją zdobytą podczas prac nad Listą 1 – tam również moja najprostsza reguła opierała się niemal w całości na przekroczeniu przez diament masy jednego karata. Różnica polega na tym, że model matematyczny sam „odkrył” tę granicę poprzez analizę statystyczną, zamiast opierać się na mojej ocenie eksperckiej.



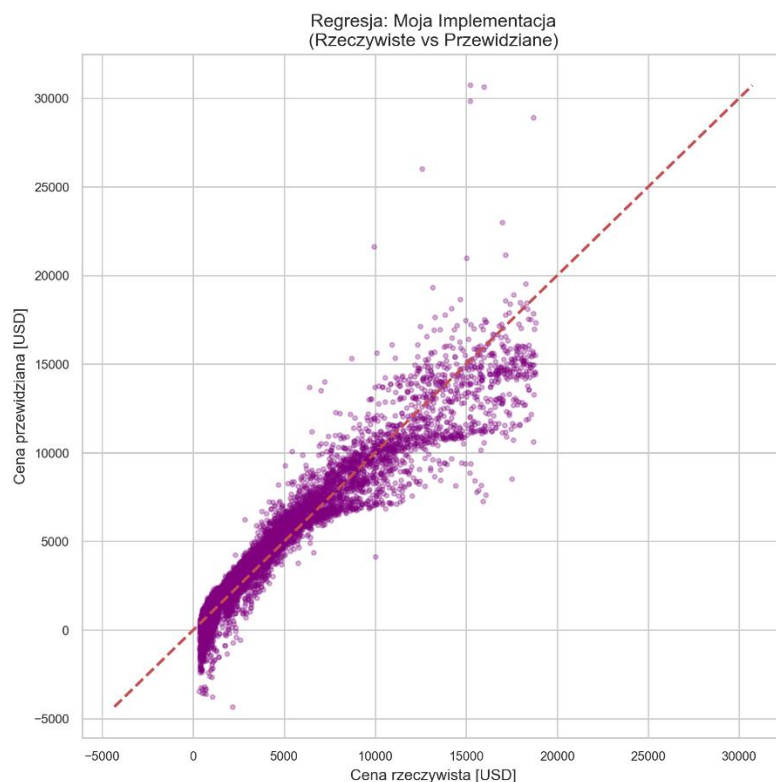
W celu weryfikacji tego wyniku, zaimplementowano funkcje obliczające Entropię oraz Zysk Informacyjny (Information Gain) z użyciem biblioteki NumPy. Ręczne obliczenie zysku informacyjnego dla zmiennej `carat` przy optymalnym progu podziału wyniosło **0.4846**. Oznacza to znaczny spadek "niepewności" (entropii) w systemie na skutek tego jednego warunku. Obliczenia potwierdziły mechanizm działania drzewa scikit-learn, dla którego parametr ten wykazywał najwyższy priorytet.

```
Ręczny Information Gain dla 'carat' przy progu 1.0: 0.4846
Feature Importance ze scikit-learn dla 'carat': 0.7772
```

4. Regresja Liniowa: Autorska Implementacja i Wagi

Kolejnym zadaniem było rozwiązanie problemu regresji – bezpośredniego przewidywania ceny diamentów. W tym celu zaimplementowano od zera model Regresji Liniowej w języku Python, oferujący dwie metody optymalizacji:

1. Rozwiązanie analityczne (Metoda Najmniejszych Kwadratów, wykorzystująca macierz pseudoodwrotną).
2. Rozwiązanie iteracyjne (Algorytm Spadku Gradientu).



Wyniki błędu średniokwadratowego (MSE) na zbiorze testowym:

```
Błąd MSE (Moja Implementacja Analityczna): 1,288,705.48
Błąd MSE (Scikit-Learn LinearRegression): 1,288,705.48
```

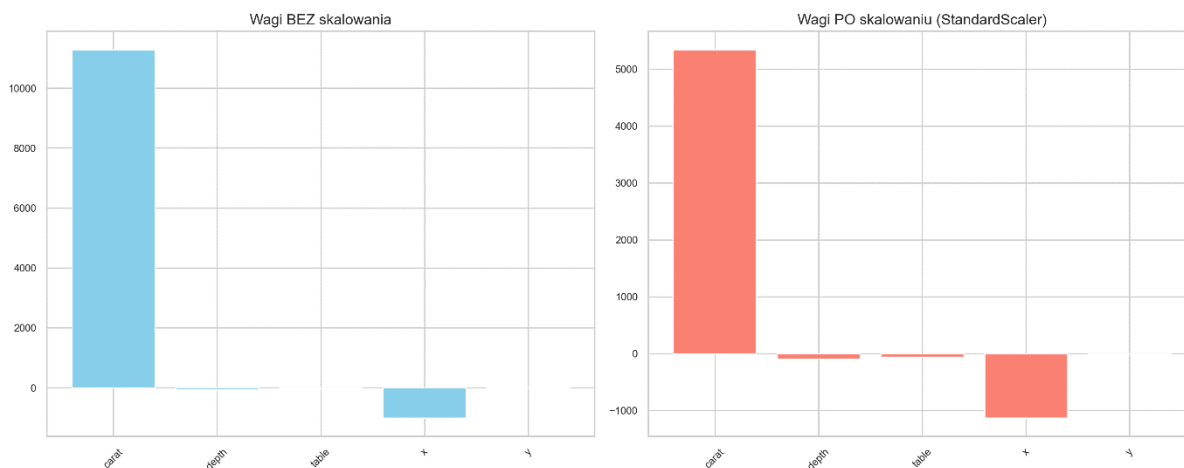
Uzyskanie identycznego wyniku (oraz tych samych wartości wyuczonych wag wektora, np. $W_0 = 3939.49$ oraz $W_1 = 5339.60$) jest ostatecznym dowodem na to, że zastosowane operacje matematyczne (mnożenie transponowanych macierzy) zostały zaprogramowane w pełni poprawnie. Wersja ze spadkiem gradientu osiągnęła błąd minimalnie różniący się, co jest naturalne dla przybliżonych algorytmów iteracyjnych.

```
Własna implementacja regresji liniowej
Porównanie wyuczonych wag (intercept + waga 'carat'):
Analitycznie (NumPy):      W0=3939.49, W1(carat)=5339.60
Spadek Gradientu (NumPy): W0=3939.49, W1(carat)=5278.01
Scikit-Learn (Gotowiec):  W0=3939.49, W1(carat)=5339.60
```

4.1. Analiza wag modelu i wpływ standaryzacji

Niezwykle ważnym aspektem modelowania jest poprawne zinterpretowanie wyuczonych parametrów. W celu zobrazowania tego problemu, zestawiono wagi cech modelu wytrenowanego na oryginalnych (surowych) danych, z wagami z modelu wykorzystującego narzędzie `StandardScaler`.

```
Interpretacja regresji przed i po skalowaniu
Waga dla cechy 'carat' BEZ SKALOWANIA: 11280.78 USD
Waga dla cechy 'carat' PO SKALOWANIU: 5339.60 USD
(Skalowanie pozwala na sprawiedliwe porównywanie wag między różnymi cechami!)
```

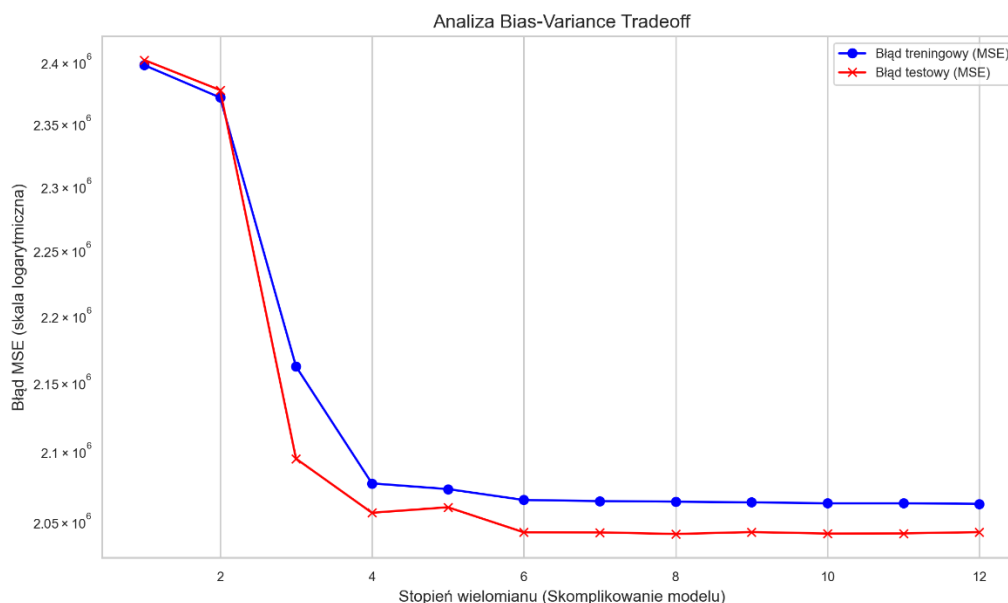


Przed skalowaniem: Model przydzielił cenie carat wagę w wysokości ok. 11 280 USD. W bezpośredniej interpretacji oznacza to, że wzrost masy diamentu o 1 fizyczny karat (przy pozostałych czynnikach niezmiennych) powoduje wzrost wartości diamentu o ponad 11 tysięcy dolarów. Choć liczba ta wydaje się logiczna, bezpośrednie porównywanie jej z wagami dla innych zmiennych (np. waga dla przejrzystości wynosząca zaledwie 30) daje fałszywy obraz rzeczywistości. Model sztucznie powiększa wagę dla karata, aby zrekompensować jego małą wartość numeryczną (od 0 do 5) w zderzeniu z ogromnymi wartościami ceny.

Po skalowaniu: Parametr carat otrzymał ustandaryzowaną wagę ok. 5 339 USD. Skalowanie odjęło średnią i znormalizowało wariancję wszystkich cech. Dzięki temu, wagi stały się "bezzjednostkowe" i obiektywne. Wykres wyraźnie pokazuje, że po standaryzacji, wpływ poszczególnych atrybutów jakości i wielkości jest o wiele bardziej proporcjonalny, co zapobiega błędnemu wyciąganiu wniosków o rzekomej "nieistotności" pozostałych kolumn.

5. Kompromis Bias-Variance oraz Błędy Ekstrapolacji

Ostatnia faza laboratorium dotyczyła analizy zachowania modeli w środowisku sztucznie kreowanych, ekstremalnych hiperparametrów, co miało na celu wywołanie stanów *Underfittingu* (niedouczenia) oraz *Overfittingu* (przeuczenia). Użyto w tym celu transformacji wielomianowych dla zmiennej `carat`.



Zgodnie z koncepcją Bias-Variance Tradeoff, na wygenerowanym wykresie możemy zaobserwować trzy strefy:

1. **High Bias (Wysokie obciążenie):** Dotyczy wielomianu stopnia 1. Model jest zbyt sztywny, przez co zarówno błąd na zbiorze treningowym (niebieski), jak i testowym (czerwony) są wysokie.
2. **Sweet Spot (Złoty środek):** Widoczny przy stopniach 2-4. Błąd testowy osiąga tam minimum, ponieważ model optymalnie generalizuje zjawisko rynkowe.
3. **High Variance (Wysoka wariancja):** Powyżej 8. stopnia, błąd treningowy stabilnie maleje niemal do zera, podczas gdy błąd testowy gwałtownie ucieka w nieskończoność (co zostało ujęte na wykresie skalą logarytmiczną). Elastyczna funkcja modelu wielomianowego zaczyna "uczyć się na pamięć" szumu z danych treningowych, ignorując rzeczywisty trend i fałując pomiędzy punktami.

5.1. Problem „Czarnego Łabędzia”

Konsekwencje overfittingu najlepiej widoczne są w problemie ekstrapolacji – zjawisku tzw. „Czarnego Łabędzia” (zdarzenia wysoce nieprawdopodobnego i nieobecnego w danych treningowych). Do testu wykorzystano anomalię: diament o masie **15 karatów**.

Eksperyment: Przewidywanie ceny diamentu o ogromnej masie 15 Karatów
 Predykcja (Zwykły model liniowy): 114,271.75 USD
 Predykcja (Przeuczony Wielomian st. 11): 1,493,105,237,490.29 USD
 Wniosek: Przeuczony model traci zdolność generalizacji (Czarne łabędzie!).

Zjawisko to potwierdza koncepcję, według której silnie wielomianowe funkcje poza zbadanym marginesem wpadają w potężne wibracje i "eksplodują", zwracając

absurdalne, bezużyteczne liczby. Aby temu zapobiec, konieczna jest regularyzacja złożoności algorytmów. W drzewach decyzyjnych stosuje się restrykcje strukturalne (np. `max_depth`), ucinające liście. Z kolei w regresji liniowej można zaimplementować modele Lasso (L1) lub Ridge (L2), które narzucają matematyczną "karę" za wysokie wartości wag wektora, powstrzymując funkcję przed drastycznymi oscylacjami.

6. Wnioski

Część II projektu wyraźnie podkreśla znaczącą przewagę modeli Machine Learning nad regułami manualnymi, przy jednoczesnym uwydatnieniu ich kruchości matematycznej. Przeprowadzone analizy pozwoliły dowieść, że:

1. Zrozumienie mechaniki "pod spodem" (entropia, zysk informacyjny) umożliwia zweryfikowanie, w jaki sposób model ocenia rzeczywistość (drzewa kategoryzują dane ostrymi podziałami przestrzeni).
2. Skalowanie danych jest krokiem absolutnie krytycznym, bez którego analiza wektorów optymalizacyjnych (regresji liniowej) traci wiarygodność.
3. Modele ulegają przeuczeniu znacznie szybciej, niż mogłoby się wydawać, co udowadnia wykres kompromisu obciążenie-wariancja. Niezabezpieczone funkcje stają się bardzo podatne na ekstrapolację, rodząc katastrofalne w skutkach błędy w zderzeniu z realnym, nieoczekiwanym zdarzeniem.

Zaimplementowany system stanowi teraz pełnowymiarowy, zautomatyzowany silnik analityczny, świadomy matematycznych zalet oraz obciążeń wykorzystanych klasyfikatorów i estymatorów.